

Logistic Regression — Classification & Evaluation Metrics

By the end of today you will understand why classification is fundamentally different from regression, how Logistic Regression works using the sigmoid function to produce probabilities, and how to evaluate classifiers using the full suite of metrics: confusion matrix, accuracy, precision, recall, F1, and ROC-AUC. You will build binary and multi-class classifiers on real datasets and read the results like a professional.

SECTION 01 Classification vs Regression — A Fundamental Difference

You have now mastered regression: predicting a continuous number. Classification is a different problem entirely. Instead of predicting how much, you predict which category. These require completely different algorithms and completely different evaluation metrics.

	Regression	Classification
Question	How much / how many?	Which category / class?
Output	Continuous number	Discrete class label
Examples	House price, temperature, salary	Spam/not spam, fraud/legit, cat/dog
Baseline model	DummyRegressor (predict mean)	DummyClassifier (predict majority)
Primary metric	RMSE, R ² , MAE	Accuracy, Precision, Recall, F1, AUC
sklearn class	LinearRegression, Ridge...	LogisticRegression, DecisionTree...

Why can't we just use Linear Regression for classification?

Linear Regression predicts any number: -5, 0, 0.3, 1.7, 100.
For binary classification we need exactly 0 or 1 (or a probability between 0 and 1).

If we try $y = w \cdot x + b$ for classification:

- For x very large: y could be 100. What class is 100?
- For x very small: y could be -50. What class is -50?
- Predictions go outside the $[0, 1]$ range and cannot be interpreted as probability.

We need a function that maps any real number to a value between 0 and 1.
That function is the sigmoid. That model is Logistic Regression.

SECTION 02 The Sigmoid Function — The Heart of Logistic Regression

The sigmoid function is what makes Logistic Regression a classifier. It takes any real number and squashes it into a probability between 0 and 1.

```
Linear Regression:    y = w*x + b           Range: (-inf, +inf)
Logistic Regression:  p = sigmoid(w*x + b)   Range: (0, 1)

The sigmoid function:
    sigmoid(z) = 1 / (1 + e^(-z))

z = -10    -> sigmoid(-10) = 0.0000454    (almost certainly class 0)
z = -2     -> sigmoid(-2)  = 0.1192       (likely class 0)
z = 0      -> sigmoid(0)   = 0.5000       (equal probability, decision
boundary)
z = +2     -> sigmoid(+2)  = 0.8808       (likely class 1)
z = +10    -> sigmoid(+10) = 0.9999546    (almost certainly class 1)

Shape: an S-curve that asymptotes at 0 and 1, crosses 0.5 at z=0.
```

```
import numpy as np
import matplotlib.pyplot as plt

# — Sigmoid function —————
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-10, 10, 300)
p = sigmoid(z)

# Show key values
print("z value | sigmoid(z) | Interpretation")
print("-" * 55)
for z_val in [-10, -5, -2, -1, 0, 1, 2, 5, 10]:
    prob = sigmoid(z_val)
    if prob < 0.1:
        interp = "Almost certainly class 0"
    elif prob < 0.4:
        interp = "Likely class 0"
    elif prob < 0.6:
        interp = "Uncertain (near boundary)"
    elif prob < 0.9:
```

```

        interp = "Likely class 1"
    else:
        interp = "Almost certainly class 1"
    print(f"{z_val:8.0f} | {prob:.6f} | {interp}")

# Plot the S-curve
plt.figure(figsize=(10, 5))
plt.plot(z, p, color='#2196F3', lw=3, label='sigmoid(z)')
plt.axhline(y=0.5, color='red', ls='--', lw=1.5, label='Decision boundary (p=0.5)')
plt.axvline(x=0, color='gray', ls='--', lw=1)
plt.fill_between(z, 0.5, p, where=(p>=0.5), alpha=0.15, color='green', label='Predict class 1')
plt.fill_between(z, p, 0.5, where=(p<0.5), alpha=0.15, color='red', label='Predict class 0')
plt.title('The Sigmoid Function - S-Curve', fontsize=14, fontweight='bold')
plt.xlabel('z = w*x + b (linear combination of features)')
plt.ylabel('Probability of class 1')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('sigmoid.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

z value		sigmoid(z)		Interpretation
-10		0.000045		Almost certainly class 0
-5		0.006693		Almost certainly class 0
-2		0.119203		Likely class 0
-1		0.268941		Likely class 0
0		0.500000		Uncertain (near boundary)
1		0.731059		Likely class 1
2		0.880797		Likely class 1
5		0.993307		Almost certainly class 1
10		0.999955		Almost certainly class 1

SECTION 03 How Logistic Regression Works

Logistic Regression

What it is: A classification algorithm that uses the sigmoid function to model the probability that a sample belongs to a particular class. Despite the name, it is a classifier, not a regressor. It outputs a probability between 0 and 1, then applies a threshold (default 0.5) to make a class prediction.

Real-life analogy: Think of a doctor deciding whether to recommend surgery. They look at multiple factors (age, blood pressure, cholesterol, test results), combine them into a risk score, and then convert that score to a 'surgery recommended or not' decision using a

threshold. Logistic Regression does exactly this: combines features into a score, converts it to a probability, then decides yes or no.

In Python: `from sklearn.linear_model import LogisticRegression`

Logistic Regression step by step:

```
Step 1: Compute linear combination  z = w1*x1 + w2*x2 + ... + wn*xn + b
Step 2: Apply sigmoid                p = 1 / (1 + e^(-z))    [now between 0
and 1]
Step 3: Apply threshold              if p >= 0.5: predict class 1
                                    if p < 0.5: predict class 0
```

During training:

Instead of MSE, uses Binary Cross-Entropy loss (log loss):

Loss = $-\text{mean}(y \cdot \log(p) + (1-y) \cdot \log(1-p))$

This penalises confident wrong predictions very heavily.

Optimised using gradient descent (same as always).

SECTION 04 Binary Classification — Your First Logistic Regression

We start with the simplest case: two classes. Is this email spam or not? Is this transaction fraud or legitimate? Is this tumour malignant or benign?

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)
from sklearn.datasets import load_breast_cancer

# — Load data: Breast cancer diagnosis —————
# 569 patients, 30 features, target: 0=malignant, 1=benign
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print(f'Dataset: {X.shape[0]} samples, {X.shape[1]} features')
print(f'Classes: {data.target_names}')
print(f'Class 0 (malignant): {np.sum(y==0)}')
print(f'Class 1 (benign) : {np.sum(y==1)}')

# — Split and scale —————
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# — Train Logistic Regression —————
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_s, y_train)

# — Predict —————
y_pred = model.predict(X_test_s)
y_pred_proba = model.predict_proba(X_test_s)[:, 1] # probability of class 1

# — Evaluate —————
print(f'\nAccuracy : {accuracy_score(y_test, y_pred):.4f}')
print(f'Precision : {precision_score(y_test, y_pred):.4f}')
print(f'Recall : {recall_score(y_test, y_pred):.4f}')
print(f'F1 Score : {f1_score(y_test, y_pred):.4f}')

print(f'\nFull Classification Report:')
print(classification_report(y_test, y_pred,
    target_names=['Malignant', 'Benign']))

# — Probabilities for first 5 test samples —————
print('Sample predictions with probabilities:')
for i in range(5):
    prob = y_pred_proba[i]
    pred = y_pred[i]
    actual = y_test.iloc[i]
    label = data.target_names[pred]
    print(f' Sample {i+1}: P(benign)={prob:.3f} -> Predicted={label}',
        f'-> Actual={data.target_names[actual]}')

```

Output:

```

Dataset: 569 samples, 30 features
Classes: ['malignant' 'benign']
Class 0 (malignant): 212
Class 1 (benign) : 357

```

```

Accuracy : 0.9737
Precision : 0.9737
Recall : 0.9861
F1 Score : 0.9799

```

Full Classification Report:

	precision	recall	f1-score	support
Malignant	0.97	0.95	0.96	42
Benign	0.97	0.99	0.98	72
accuracy			0.97	114

Sample predictions with probabilities:

```
Sample 1: P(benign)=0.998 -> Predicted=benign    -> Actual=benign
Sample 2: P(benign)=0.023 -> Predicted=malignant -> Actual=malignant
Sample 3: P(benign)=0.987 -> Predicted=benign    -> Actual=benign
Sample 4: P(benign)=0.421 -> Predicted=malignant -> Actual=benign <-
uncertain!
Sample 5: P(benign)=0.001 -> Predicted=malignant -> Actual=malignant
```

97.4% accuracy on cancer diagnosis with 30 features. That is excellent.

Notice Sample 4: $P(\text{benign})=0.421$ predicted as malignant.

The model was uncertain (close to 0.5) and chose malignant.

In cancer screening this is the RIGHT choice — when uncertain, flag for review.

This is where the threshold becomes important:

Default threshold = 0.5: balanced precision and recall.

Lower threshold (e.g. 0.3): catch more cancers (higher recall) but more false alarms.

The medical team would choose the threshold based on clinical judgement.

SECTION 05 The Confusion Matrix — The Full Picture of Performance

Accuracy tells you what percentage of predictions were correct. But it hides the important details: which type of errors did the model make? The confusion matrix reveals everything.

		PREDICTED		
		Class 0	Class 1	
ACTUAL	Class 0	TN	FP	<- Actual Negatives
	Class 1	FN	TP	<- Actual Positives

```
TN = True Negative   : Predicted 0, Actual 0  (correct: said No, was No)
FP = False Positive  : Predicted 1, Actual 0  (wrong: said Yes, was No)  =
Type I Error
FN = False Negative  : Predicted 0, Actual 1  (wrong: said No, was Yes)  =
Type II Error
TP = True Positive   : Predicted 1, Actual 1  (correct: said Yes, was Yes)
```

From these four numbers, all other metrics are derived.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
```

```

# Using the breast cancer predictions from Section 04
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()

print('Confusion Matrix:')
print(cm)
print(f'\nTN={tn}  FP={fp}  FN={fn}  TP={tp}')

# Compute all metrics from scratch
accuracy = (tp + tn) / (tp + tn + fp + fn)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)
specificity = tn / (tn + fp) # True Negative Rate

print(f'\nFrom the confusion matrix:')
print(f'  Accuracy    = (TP+TN)/Total = ({tp}+{tn})/({tp}+{tn}+{fp}+{fn}) = {accuracy:.4f}')
print(f'  Precision    = TP/(TP+FP)    = {tp}/({tp}+{fp})          = {precision:.4f}')
print(f'  Recall        = TP/(TP+FN)    = {tp}/({tp}+{fn})          = {recall:.4f}')
print(f'  F1 Score       = 2*P*R/(P+R)    = {f1:.4f}')
print(f'  Specificity    = TN/(TN+FP)    = {tn}/({tn}+{fp})          = {specificity:.4f}')

# Visualise with heatmap
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Malignant', 'Benign'],
            yticklabels=['Malignant', 'Benign'],
            linewidths=2, linecolor='white')
plt.title('Confusion Matrix - Breast Cancer Classifier',
          fontsize=13, fontweight='bold')
plt.ylabel('Actual Class', fontsize=12)
plt.xlabel('Predicted Class', fontsize=12)
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

Confusion Matrix:

```
[[40  2]
 [ 1 71]]
```

TN=40 FP=2 FN=1 TP=71

From the confusion matrix:

```

Accuracy    = (TP+TN)/Total = (71+40)/114 = 0.9737
Precision    = TP/(TP+FP)    = 71/(71+2)   = 0.9726
Recall       = TP/(TP+FN)    = 71/(71+1)   = 0.9861
F1 Score     = 2*P*R/(P+R)   =              = 0.9793
Specificity  = TN/(TN+FP)    = 40/(40+2)   = 0.9524

```

Reading this confusion matrix for cancer diagnosis:

TN=40: 40 malignant tumours correctly identified as malignant.

FP=2 : 2 malignant tumours incorrectly called benign. DANGEROUS. These patients might not get treatment they need.

FN=1 : 1 benign tumour incorrectly called malignant. Unnecessary worry/treatment.

TP=71: 71 benign tumours correctly identified as benign.

For cancer: FP=2 is MORE dangerous than FN=1.

Missing a cancer (FP) is worse than a false alarm (FN).

We want to MINIMISE false positives -> maximise RECALL (catch all real cancers).

This is why you always interpret the confusion matrix in domain context.

SECTION 06 All Classification Metrics Explained

There are five core metrics for classification. Each answers a different question. Know all five, know when each matters, and know how to explain them to a non-technical audience.

Metric 1 — Accuracy

What fraction of all predictions were correct? Simple but can be misleading when classes are imbalanced.

```
# Accuracy is misleading with imbalanced classes
import numpy as np
from sklearn.metrics import accuracy_score

# 1000 transactions: 990 legitimate, 10 fraud
y_true = np.array([0]*990 + [1]*10)

# Model that ALWAYS predicts legitimate (no fraud detected ever)
y_lazy = np.zeros(1000, dtype=int)

# Real fraud detector (catches 8 of 10 frauds)
y_real = np.array([0]*990 + [0,0,1,1,1,1,1,1,1,1])

print(f'Lazy model accuracy: {accuracy_score(y_true, y_lazy):.2%}')
print(f'Real model accuracy: {accuracy_score(y_true, y_real):.2%}')
print(f'Frauds caught by lazy model: 0 out of 10')
print(f'Frauds caught by real model: 8 out of 10')
print(f'\nAccuracy alone makes the lazy model look equally good!')
print(f'Always use Precision + Recall for imbalanced problems.')
```

Output:

```
Lazy model accuracy: 99.00%
Real model accuracy: 99.80%
Frauds caught by lazy model: 0 out of 10
Frauds caught by real model: 8 out of 10
```

Accuracy alone makes the lazy model look equally good!
Always use Precision + Recall for imbalanced problems.

Metrics 2 & 3 — Precision and Recall

Precision and recall answer complementary questions. You almost always need both together.

Metric	Formula	Plain English Question
Precision	$TP / (TP + FP)$	Of all the things I labelled as positive, how many actually were?
Recall	$TP / (TP + FN)$	Of all the actual positives, how many did I catch?

```
from sklearn.metrics import precision_score, recall_score
import numpy as np

# Scenario: Email spam filter
# 1=spam, 0=legitimate
y_true = np.array([1,1,1,1,1,0,0,0,0,0,0,0,0,0,0])
# 5 spam, 10 legitimate

# Model A: aggressive (catches all spam but also blocks some real emails)
y_pred_a = np.array([1,1,1,1,1,1,1,1,0,0,0,0,0,0,0])
# Caught all 5 spam + blocked 3 real emails

# Model B: conservative (only blocks very obvious spam)
y_pred_b = np.array([1,1,1,0,0,0,0,0,0,0,0,0,0,0,0])
# Caught 3 spam, blocked no real emails

for name, y_pred in [('Model A (aggressive)', y_pred_a),
                     ('Model B (conservative)', y_pred_b)]:
    p = precision_score(y_true, y_pred)
    r = recall_score(y_true, y_pred)
    print(f'{name}:')
    print(f' Precision = {p:.2f}   (of flagged emails, {p*100:.0f}% truly spam)')
    print(f' Recall    = {r:.2f}   (of actual spam, caught {r*100:.0f}%)')
    print()
```

Output:

```
Model A (aggressive):
 Precision = 0.63   (of flagged emails, 63% truly spam)
 Recall    = 1.00   (of actual spam, caught 100%)
```

```
Model B (conservative):  
  Precision = 1.00   (of flagged emails, 100% truly spam)  
  Recall    = 0.60   (of actual spam, caught 60%)
```

The precision-recall trade-off in plain English:

Model A: Catches every spam (100% recall) but also blocks 3 real emails.
Your boss's email might get blocked. High recall, low precision.

Model B: Never blocks a real email (100% precision) but misses 2 spam emails.
Some spam gets through. High precision, low recall.

Neither is universally better. The BUSINESS CONTEXT decides:

Spam filter: prefer precision (do not block real emails)

Cancer detection: prefer recall (do not miss cancer)

Fraud detection: prefer recall (do not miss fraud)

Job application screener: prefer precision (only invite strong candidates)

Metric 4 — F1 Score

F1 is the harmonic mean of precision and recall. It gives a single number that balances both. It is especially useful when classes are imbalanced and you cannot favour either precision or recall.

```
from sklearn.metrics import f1_score  
import numpy as np  
  
# Why harmonic mean instead of arithmetic mean?  
p1, r1 = 1.0, 0.01   # Model with 100% precision but only 1% recall  
p2, r2 = 0.6, 0.6    # Model with balanced 60% on both  
  
# Arithmetic mean would give same score to both:  
arith1 = (p1 + r1) / 2  
arith2 = (p2 + r2) / 2  
  
# Harmonic mean penalises extreme imbalances:  
harm1 = 2 * p1 * r1 / (p1 + r1)  
harm2 = 2 * p2 * r2 / (p2 + r2)  
  
print('Model 1: Precision=1.00, Recall=0.01')  
print(f'  Arithmetic mean: {arith1:.2f}  <- falsely looks decent')  
print(f'  Harmonic mean (F1): {harm1:.4f}  <- reveals it is almost  
useless')  
print()  
print('Model 2: Precision=0.60, Recall=0.60')  
print(f'  Arithmetic mean: {arith2:.2f}')  
print(f'  Harmonic mean (F1): {harm2:.4f}  <- a genuinely balanced model')
```

```
print()
print('F1 punishes models that excel at one metric while ignoring the
other.')
```

Output:

```
Model 1: Precision=1.00, Recall=0.01
  Arithmetic mean: 0.51  <- falsely looks decent
  Harmonic mean (F1): 0.0198  <- reveals it is almost useless

Model 2: Precision=0.60, Recall=0.60
  Arithmetic mean: 0.60
  Harmonic mean (F1): 0.6000  <- a genuinely balanced model

F1 punishes models that excel at one metric while ignoring the other.
```

Metric 5 — ROC Curve and AUC

The ROC (Receiver Operating Characteristic) curve and AUC (Area Under the Curve) measure model performance across all possible thresholds. This is the most complete single metric for a binary classifier.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load and prepare data
data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)
sc = StandardScaler()
X_tr = sc.fit_transform(X_tr)
X_te = sc.transform(X_te)

model = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
y_proba = model.predict_proba(X_te)[:, 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_te, y_proba)
auc = roc_auc_score(y_te, y_proba)

print(f'ROC-AUC Score: {auc:.4f}')
print(f'\nAUC interpretation:')
print(f'  AUC = 0.50  -> Random guessing (no better than coin flip)')
print(f'  AUC = 0.70  -> Acceptable discriminating ability')
print(f'  AUC = 0.80  -> Good discriminating ability')
```

```

print(f'  AUC = 0.90  -> Excellent discriminating ability')
print(f'  AUC = 1.00  -> Perfect classifier')
print(f'  Our model  -> {auc:.4f} = {"Excellent" if auc>0.9 else "Good"}')

# Plot
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='#2196F3', lw=2.5,
         label=f'Logistic Regression (AUC = {auc:.3f})')
plt.plot([0,1],[0,1], color='gray', lw=1.5, ls='--', label='Random (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.15, color='#2196F3')
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
plt.ylabel('True Positive Rate (Recall)', fontsize=12)
plt.title('ROC Curve - Breast Cancer Classifier', fontsize=13,
          fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('roc_curve.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

ROC-AUC Score: 0.9967

AUC interpretation:

```

AUC = 0.50  -> Random guessing (no better than coin flip)
AUC = 0.70  -> Acceptable discriminating ability
AUC = 0.80  -> Good discriminating ability
AUC = 0.90  -> Excellent discriminating ability
AUC = 1.00  -> Perfect classifier
Our model  -> 0.9967 = Excellent

```

What the ROC curve shows:

X-axis: False Positive Rate (FPR) = $FP / (FP + TN)$

How often we wrongly say 'positive' when it is actually negative.

Y-axis: True Positive Rate (TPR) = Recall = $TP / (TP + FN)$

How often we correctly catch actual positives.

The curve plots TPR vs FPR at every possible threshold (0.0 to 1.0).

A perfect model: immediately goes to top-left corner (TPR=1, FPR=0).

A random model: follows the diagonal line (AUC=0.5).

AUC = probability that the model ranks a random positive sample higher than a random negative sample.

Binary classification is common but many real problems have more than two classes. Logistic Regression handles this automatically using a technique called one-vs-rest (OvR): it trains one binary classifier per class and picks the class with the highest confidence.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.datasets import load_iris
import seaborn as sns
import matplotlib.pyplot as plt

# — Iris: 3-class classification —————
iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

sc = StandardScaler()
X_train_s = sc.fit_transform(X_train)
X_test_s = sc.transform(X_test)

# multi_class='ovr' = One vs Rest (default for many solvers)
model = LogisticRegression(max_iter=1000, random_state=42,
                           multi_class='ovr')
model.fit(X_train_s, y_train)

y_pred = model.predict(X_test_s)

print('=== Multi-class Classification Report ===')
print(classification_report(y_test, y_pred,
                            target_names=iris.target_names))

# Confusion matrix for 3 classes
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names,
            linewidths=2, linecolor='white')
plt.title('Confusion Matrix - Iris 3-Class', fontsize=13,
          fontweight='bold')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.savefig('multiclass_cm.png', dpi=150, bbox_inches='tight')
```

```
plt.show()

# Predict probabilities for all 3 classes
proba = model.predict_proba(X_test_s[:3])
print('\nClass probabilities for first 3 test samples:')
for i, row in enumerate(proba):
    print(f' Sample {i+1}: ',
          f'setosa={row[0]:.3f}, versicolor={row[1]:.3f},
          virginica={row[2]:.3f}',
          f'-> Predicted: {iris.target_names[np.argmax(row)]}')

```

Output:

```
=== Multi-class Classification Report ===
      precision  recall  f1-score  support
setosa         1.00    1.00    1.00        10
versicolor     1.00    0.90    0.95        10
virginica       0.91    1.00    0.95        10
accuracy                    0.97        30
macro avg       0.97    0.97    0.97        30

Class probabilities for first 3 test samples:
Sample 1: setosa=0.998, versicolor=0.002, virginica=0.000 -> Predicted:
setosa
Sample 2: setosa=0.000, versicolor=0.982, virginica=0.018 -> Predicted:
versicolor
Sample 3: setosa=0.000, versicolor=0.039, virginica=0.961 -> Predicted:
virginica

```

SECTION 08 Threshold Tuning — Adjusting the Decision Boundary

The default threshold of 0.5 is not always the best. In many real problems, you deliberately adjust it based on the business cost of different types of errors.

```
import numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score

# Using the breast cancer probabilities from Section 04
# y_pred_proba = model.predict_proba(X_test_s)[: , 1]
# (Run Section 04 code first)

thresholds = [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

print(f"{'Threshold':>10} | {'Precision':>10} | {'Recall':>8} | ",
      f"{'F1':>8} | {'Predicted+':>11} | Note")
print("-" * 85)

for t in thresholds:
    y_pred_t = (y_pred_proba >= t).astype(int)

```

```

p = precision_score(y_test, y_pred_t, zero_division=0)
r = recall_score(y_test, y_pred_t)
f1 = f1_score(y_test, y_pred_t, zero_division=0)
n_pos = y_pred_t.sum()

note = ''
if t == 0.5:
    note = '<- default'
elif t < 0.4:
    note = '<- high recall, more false alarms'
elif t > 0.6:
    note = '<- high precision, misses more cancers'

print(f"{t:>10.1f} | {p:>10.4f} | {r:>8.4f} | {f1:>8.4f} | ",
      f"{n_pos:>11} | {note}")

print(f'\nFor cancer detection: choose threshold = 0.3 or lower.')
print(f'Catching every cancer (recall) matters more than false alarms.')

```

Output:

Threshold	Precision	Recall	F1	Predicted+	Note
0.2	0.9481	1.0000	0.9733	76	<- high recall, more false alarms
0.3	0.9589	1.0000	0.9790	74	<- high recall, more false alarms
0.4	0.9726	1.0000	0.9861	73	
0.5	0.9726	0.9861	0.9793	72	<- default
0.6	0.9857	0.9583	0.9718	70	
0.7	1.0000	0.9444	0.9714	68	<- high precision, misses more
0.8	1.0000	0.9028	0.9490	65	<- high precision, misses more

For cancer detection: choose threshold = 0.3 or lower.
Catching every cancer (recall) matters more than false alarms.

SECTION 09 Logistic Regression Hyperparameters

Logistic Regression has several hyperparameters that control regularisation and convergence. Knowing these is essential for tuning the model in practice.

Parameter	Default	What it does
C	1.0	Inverse of regularisation strength. $C=1/\alpha$. Smaller C = stronger regularisation.

penalty	'l2'	Type of regularisation. 'l2'=Ridge, 'l1'=Lasso, 'elasticnet', None.
solver	'lbfgs'	Optimisation algorithm. 'saga' supports l1 and elasticnet.
max_iter	100	Maximum iterations. Increase if convergence warning appears.
multi_class	'auto'	'ovr' for one-vs-rest, 'multinomial' for softmax.
class_weight	None	'balanced' to handle imbalanced classes automatically.
random_state	None	Seed for reproducibility.

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)

# Grid search over C values
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100]}

grid_search = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=42),
    param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
grid_search.fit(X_tr_s, y_tr)

print(f'Best C      : {grid_search.best_params_["C"]}')
print(f'Best CV F1   : {grid_search.best_score_:.4f}')
print(f'Test F1      : {grid_search.score(X_te_s, y_te):.4f}')

# Show all C values and their CV scores
results = grid_search.cv_results_
print(f'\nAll C values vs CV F1 score:')
for c, score in zip(param_grid['C'], results['mean_test_score']):
    marker = ' <- best' if c == grid_search.best_params_['C'] else ''
    print(f'  C={c:>6}: CV F1={score:.4f}{marker}')

```

Output:

```
Best C          : 0.1
Best CV F1      : 0.9855
Test F1         : 0.9861
```

All C values vs CV F1 score:

```
C= 0.001: CV F1=0.9763
C= 0.01: CV F1=0.9810
C= 0.1: CV F1=0.9855 <- best
C= 1: CV F1=0.9840
C= 10: CV F1=0.9826
C= 100: CV F1=0.9826
```

Remember: C is the INVERSE of regularisation strength.

C = 0.001 -> Very strong regularisation (like $\alpha=1000$ in Ridge)

C = 0.1 -> Moderate regularisation

C = 1.0 -> Default, light regularisation

C = 100 -> Almost no regularisation

Best C = 0.1 means moderate regularisation works best here.

The model benefits from being slightly constrained to generalise better.

SECTION 10 Full Pipeline — End to End on Real Data

A complete, production-quality Logistic Regression pipeline. This is the template to adapt for your homework and any classification project.

```
# =====
# Logistic Regression – Complete Professional Pipeline
# =====

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report
)
from sklearn.dummy import DummyClassifier
from sklearn.datasets import load_breast_cancer
import warnings
warnings.filterwarnings('ignore')
```

```

# — Load —————
data = load_breast_cancer()
X     = pd.DataFrame(data.data, columns=data.feature_names)
y     = data.target
print(f'Dataset: {X.shape}, Classes: {data.target_names}')

# — Split —————
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# — Scale —————
sc = StandardScaler()
X_trs = sc.fit_transform(X_tr)
X_tes = sc.transform(X_te)

# — Baseline —————
dummy = DummyClassifier(strategy='most_frequent').fit(X_trs, y_tr)
print(f'Baseline accuracy: {accuracy_score(y_te,
dummy.predict(X_tes)):.4f}')

# — Train —————
model = LogisticRegression(C=0.1, max_iter=1000, random_state=42)
model.fit(X_trs, y_tr)

# — Evaluate —————
y_pred = model.predict(X_tes)
y_proba = model.predict_proba(X_tes)[:, 1]

print(f'\nLogistic Regression Results:')
print(f' Accuracy   : {accuracy_score(y_te, y_pred):.4f}')
print(f' Precision  : {precision_score(y_te, y_pred):.4f}')
print(f' Recall     : {recall_score(y_te, y_pred):.4f}')
print(f' F1 Score   : {f1_score(y_te, y_pred):.4f}')
print(f' ROC-AUC    : {roc_auc_score(y_te, y_proba):.4f}')

# — Cross-validation for robust estimate —————
from sklearn.pipeline import Pipeline
pipe = Pipeline([('scaler', StandardScaler()),
                 ('clf', LogisticRegression(C=0.1, max_iter=1000))])
cv_scores = cross_val_score(pipe, X, y, cv=5, scoring='f1')
print(f'\n5-Fold CV F1: {cv_scores.mean():.4f} (+/-
{cv_scores.std():.4f})')
print(f'CV Scores   : {cv_scores.round(4)}')

```

Output:

```

Dataset: (569, 30), Classes: ['malignant' 'benign']
Baseline accuracy: 0.6228

```

Logistic Regression Results:

```

Accuracy   : 0.9737
Precision  : 0.9726
Recall     : 0.9861

```

```
F1 Score : 0.9793
ROC-AUC : 0.9967
```

```
5-Fold CV F1: 0.9787 (+/- 0.0121)
CV Scores : [0.9714 0.9859 0.9855 0.9706 0.9797]
```

SECTION 11 Interview Prep — Questions to Answer After Today

Logistic Regression and classification metrics are tested in virtually every ML interview. These questions cover both conceptual depth and practical application.

Q — What is Logistic Regression and how is it different from Linear Regression?

Logistic Regression is a classification algorithm that predicts the probability of a sample belonging to a class. Despite the name, it is not used for regression. Linear Regression predicts a continuous number using $y = w \cdot x + b$, which can output any value from negative infinity to positive infinity. Logistic Regression applies the sigmoid function to that linear combination: $p = \text{sigmoid}(w \cdot x + b) = 1 / (1 + e^{-z})$, which squashes any input into the range 0 to 1. This output is interpreted as a probability. A threshold (default 0.5) converts the probability to a class label. Linear Regression minimises MSE. Logistic Regression minimises Binary Cross-Entropy loss, which heavily penalises confident wrong predictions.

Q — Explain precision, recall, and F1 in your own words. When would you use each?

Precision asks: of all the samples I predicted as positive, what fraction were actually positive? It measures the reliability of positive predictions. Use precision when false positives are costly, like in a spam filter where incorrectly blocking a real email is very undesirable. Recall asks: of all the actual positives in the dataset, what fraction did I correctly identify? It measures completeness. Use recall when false negatives are costly, like in cancer screening where missing a real cancer case could be fatal. F1 is the harmonic mean of precision and recall. It gives a single balanced score. Use F1 when you need to balance both concerns and when classes are imbalanced, because accuracy becomes misleading in those situations.

Q — What is ROC-AUC and why is it preferred over accuracy for imbalanced datasets?

The ROC curve plots the True Positive Rate (recall) against the False Positive Rate at every possible classification threshold from 0 to 1. AUC is the area under this curve, ranging from 0.5 (random guessing) to 1.0 (perfect classification). AUC is preferred over accuracy for imbalanced datasets because accuracy is dominated by the majority class. A model that always predicts the majority class gets high accuracy without learning anything useful. AUC measures how well the model can distinguish between classes regardless of threshold, making it class-imbalance insensitive. AUC of 0.9 means there is a 90% probability that the model ranks a randomly chosen positive sample higher than a randomly chosen negative sample.

Q — What does the C parameter in Logistic Regression control?

C is the inverse of regularisation strength. $C = 1/\alpha$ in Ridge/Lasso terminology. A small C means strong regularisation, which forces coefficients to be small and prevents overfitting. A

large C means weak regularisation, allowing the model to fit the training data more closely. The default $C=1$ applies moderate regularisation. When your model is underfitting (low training accuracy), increase C. When it is overfitting (high training, low test accuracy), decrease C. You find the optimal C using cross-validation with GridSearchCV. Logistic Regression with `penalty='l2'` applies Ridge-style regularisation by default, with `penalty='l1'` it applies Lasso-style which can zero out features.

SECTION 12 Words to Know

Term	Definition
Logistic Regression	Classification algorithm using sigmoid to output class probabilities.
Sigmoid function	S-shaped curve: maps any number to (0,1). $\text{sigmoid}(z) = 1/(1+e^{(-z)})$
Binary classification	Two-class problem: spam/not spam, fraud/legit, benign/malignant.
Multi-class classification	Three or more classes: digit recognition, species classification.
Threshold	The probability cutoff for class decision. Default 0.5.
Decision boundary	The surface separating predicted classes. In 2D, a line.
Log loss (cross-entropy)	Cost function for classification. Penalises confident wrong predictions.
<code>predict_proba()</code>	Returns class probabilities (not just the class label).
Confusion matrix	Table of TP, TN, FP, FN showing all prediction outcomes.
True Positive (TP)	Model predicted positive and it was actually positive.
False Positive (FP)	Model predicted positive but it was actually negative. Type I error.
False Negative (FN)	Model predicted negative but it was actually positive. Type II error.
True Negative (TN)	Model predicted negative and it was actually negative.
Accuracy	$(TP+TN)/\text{Total}$. Misleading for imbalanced classes.
Precision	$TP/(TP+FP)$. Reliability of positive predictions.
Recall (Sensitivity)	$TP/(TP+FN)$. Completeness of positive detection.
Specificity	$TN/(TN+FP)$. True Negative Rate.
F1 Score	$2*P*R/(P+R)$. Harmonic mean of precision and recall.
ROC Curve	TPR vs FPR at all thresholds. Shows model discrimination ability.
AUC	Area Under ROC Curve. 0.5=random, 1.0=perfect.
C parameter	Inverse regularisation strength. Small C = strong regularisation.
One-vs-Rest (OvR)	Multi-class strategy: train one binary classifier per class.
<code>class_weight='balanced'</code>	Auto-adjusts for imbalanced classes during training.
GridSearchCV	Try all hyperparameter combinations with CV to find best.

Create `day16_homework.py`. Every classification exercise must report the full set: accuracy, precision, recall, F1, and AUC. Always compare to a `DummyClassifier` baseline. Always scale features. Push to GitHub before midnight.

Exercise 1 — Titanic Survival Classifier (Beginner)

Load the cleaned Titanic dataset from Day 8. Use numeric features: `Pclass`, `Age`, `SibSp`, `Parch`, `Fare`. Target: `Survived`. Scale features. Train `LogisticRegression` (`max_iter=1000`). Print the confusion matrix and full classification report. Write 3 sentences interpreting the results: who does the model struggle to classify correctly and why might that be?

Exercise 2 — Sigmoid from Scratch (Beginner-Intermediate)

Implement the sigmoid function from scratch using only NumPy. Then implement a `predict_proba` function that takes `X`, weights `w`, and bias `b`, computes the linear combination $z = X @ w + b$, applies your sigmoid, and returns probabilities. Finally implement a `predict` function that applies `threshold=0.5`. Test it on 5 samples with known `w` and `b` values and verify the output matches sklearn's `LogisticRegression` `predict_proba` on the same data.

Exercise 3 — Precision-Recall Trade-off (Intermediate)

Using the breast cancer dataset, train `LogisticRegression` (`C=0.1`). Plot precision, recall, and F1 vs threshold (try thresholds from 0.1 to 0.9 in steps of 0.05). Mark the default threshold of 0.5. Answer in comments: (1) At what threshold is recall highest? (2) At what threshold is precision highest? (3) What threshold would you choose for cancer detection and why?

Exercise 4 — Imbalanced Classes (Intermediate)

Create an imbalanced dataset: 950 class-0 samples and 50 class-1 samples using `make_classification` with `weights=[0.95, 0.05]`. Train two logistic regression models: (1) standard `LogisticRegression`, (2) `LogisticRegression` with `class_weight='balanced'`. For each, print accuracy, precision, recall, F1, and AUC. Show that accuracy is misleading here and explain which model is better and why.

Exercise 5 — Multi-class on Wine Dataset (Intermediate)

Load `load_wine` from `sklearn.datasets`. This has 3 classes (wine cultivars). Train `LogisticRegression` (`max_iter=2000`, `C=0.5`). Print the confusion matrix as a heatmap with class names. Print the classification report. Answer: which wine class does the model classify most accurately? Which is hardest? Plot the ROC curve for each class using a one-vs-rest approach (`roc_curve` with `pos_label` parameter).

Stretch Challenge — Tier A students

Implement logistic regression from scratch using gradient descent. The gradient for binary cross-entropy is: $dW = (1/n) * X^T * (\text{sigmoid}(X @ W + b) - y)$, $db = \text{mean}(\text{sigmoid}(X @ W + b) - y)$. Train for 1000 iterations with `learning_rate=0.1`. Use the breast cancer dataset (after scaling). Compare your learned weights, accuracy, and log loss against sklearn's `LogisticRegression(C=1e10)` which applies almost no regularisation. They should produce near-identical results.